

TaurEye

Complete Technical Architecture Report

An end-to-end engineering reference for an NSE/BSE end-of-day equity **screener**: a React/TypeScript single-page application (web + Android/iOS via Capacitor) backed by a precomputed JSON data bundle produced nightly by a Python data engine. This document specifies every layer — the data pipeline, the SQLite spine and corporate-action adjustment, the JSON bundle format, the client data layer, the analytics (indicators, chart-pattern detection, natural-language screening), the UI, authentication, the mobile/OTA platform, and the full deployment topology — in enough detail to reproduce the system.

Property	Value
Product class	EOD stock screener (Indian markets: NSE / BSE, NIFTY / BANKNIFTY / F&O context)
Frontend	React 18 + TypeScript, Vite, React Router v6
Mobile	Capacitor 8 (Android + iOS), Capgo over-the-air updates
Backend / data engine	Python 3.11+, pure-stdlib indicators, SQLite spine, FastAPI (legacy)
Production model	Fully static — precomputed JSON bundle served by nginx; no live API at runtime
Auth	Supabase (GoTrue) — email/password, OAuth, PKCE; guest/offline fallback
Hosting	Single VM, Docker Compose: web (nginx) + dataengine + scheduler + self-hosted Supabase

Confidential engineering document. Status as of the current build. Code references cite file paths and line numbers from the source tree.

Contents

1. System Overview & Architecture
2. Backend Data Engine — Directory & Modules
3. Data Sources (NSE / BSE) & Bhavcopy Parsing
4. The SQLite Spine — Schema
5. Ingest Pipeline (nightly orchestration)
6. Corporate-Action Adjustment Algorithm
7. NSE/BSE Reconciliation & Multi-ISIN Merge
8. Server-Side Indicator Computation
9. The Export Step — JSON Bundle Format
10. HTTP Layer, CLI & Configuration
11. Frontend Data Layer (source switch, snapshot, client)
12. Type System & Client Screening Engine
13. Technical Indicators (formulas & periods)
14. Chart-Pattern Detection
15. Natural-Language Screener
16. Screener Filter Model & Report Building
17. Frontend UI — Routing, Shell, Pages, Components
18. Client State & Persistence; Content System
19. Authentication (Supabase)
20. Mobile Platform (Capacitor + Capgo OTA)
21. Deployment & Infrastructure (CI/CD, Docker, nginx)
22. Reproduction Blueprint & Checklist

1. System Overview & Architecture

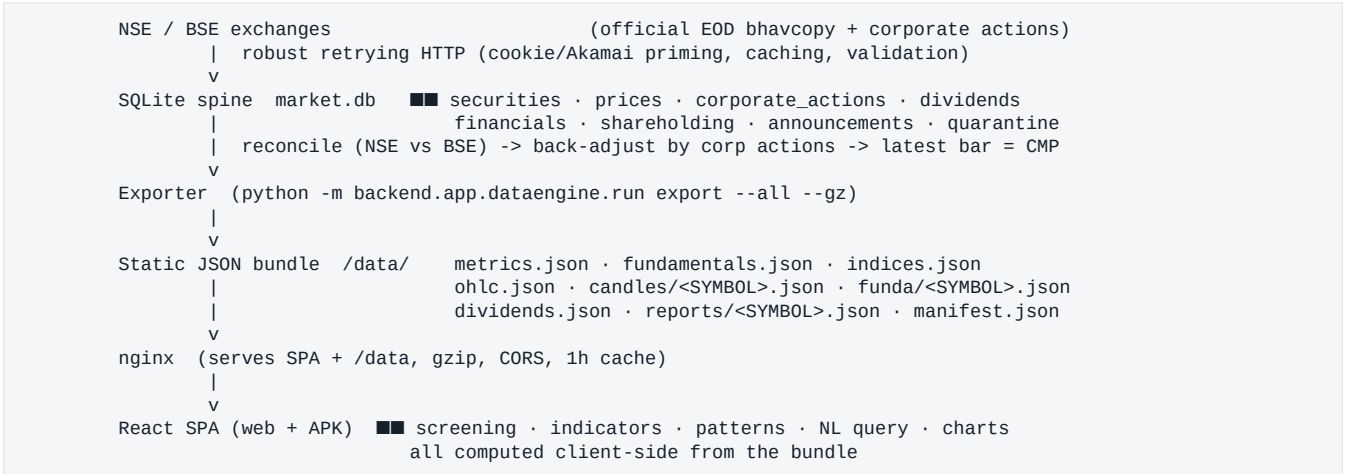
The product is a stock **screeener** for Indian equities (~5,800 NSE/BSE scrips) with technical indicators, chart-pattern detection, a natural-language query builder, watchlists, portfolios, alerts, and per-stock factual reports. It ships as one codebase to three targets: the production website, an Android APK, and iOS — all from the same React/TypeScript source.

The static hosting model (the central design decision)

Production has **no live application backend**. A nightly job on the VM pulls end-of-day (EOD) data into a large SQLite database (the “spine”, `market.db`), corporate-action-adjusts every series so the latest bar equals the current price, then **exports a static JSON bundle**. `nginx` serves the built SPA plus that bundle under `/data`. The frontend does all the work client-side: screening, indicators, pattern detection, the NL screeener, and stock detail pages all read the bundle. This makes the site cheap, fast, offline-capable, and trivially scalable.

Dual-mode data layer. A single build-time switch `VITE_DATA_SOURCE=local` sets a global `LOCAL_DATA=true`. When true, every API call resolves against the JSON bundle (this is how production web AND the APK build). When unset, the legacy mode hits a live FastAPI `/api` (retired in production). `src/api/client.ts` branches every method on this flag, so the two modes share one codebase — only environment variables differ.

End-to-end data flow



Two delivery rails keep clients fresh without redeploying: pushing to `main` rebuilds and rsyncs the SPA to `nginx` (web, ~1 min), and the same push uploads a new web bundle to Capgo so installed apps update over the air on next launch. **Data** freshness is owned by the nightly cron, independent of code deploys.

2. Backend Data Engine — Directory & Modules

The engine lives under `backend/`. It is pure Python (3.11+) with a deliberately small dependency set so it can run unattended on a small VM.

Module	Responsibility
<code>app/main.py</code>	Legacy FastAPI app (REST endpoints) — retired in production, retained for parity.
<code>app/models.py</code>	Pydantic schemas: Security, Candle, Metrics, ScreenRequest/Response, FieldDef.
<code>app/indicators.py</code>	Pure-Python technical indicators (SMA, EMA, RSI, MACD, ATR) + signal flags.
<code>app/screener.py</code>	FIELDS catalog + filter/sort engine (the server twin of the client screener).
<code>app/indices.py</code>	Live market-index ticker (NIFTY, BANKNIFTY, SENSEX, VIX, FX, globals).
<code>app/store.py</code>	In-memory Metrics snapshot cache for the live API.
<code>app/ai.py / report.py</code>	Templated per-symbol factual report generation.
<code>app/push.py</code>	Firebase Cloud Messaging broadcast when <code>data_date</code> advances.
<code>app/providers/</code>	<code>base.py</code> interface; <code>db.py</code> reads the spine; <code>synthetic.py</code> demo generator.
<code>dataengine/config.py</code>	URLs, paths, HTTP tuning, reconciliation tolerances, env var handling.
<code>dataengine/types.py</code>	Frozen dataclasses: <code>MasterRow</code> , <code>PriceRow</code> , <code>CorpAction</code> .
<code>dataengine/db.py</code>	SQLite schema (CREATE TABLE) + migrations.
<code>dataengine/http.py</code>	Retrying HTTP with cookie/Akamai priming, caching, byte validation.
<code>dataengine/sources.py</code>	Exchange adapters: NSE/BSE bhavcopy parsers, corp-action feeds.
<code>dataengine/ingest.py</code>	Orchestration: master -> corp actions -> EOD -> readjust -> segments/sectors.
<code>dataengine/reconcile.py</code>	Cross-verify NSE vs BSE prices; quarantine disagreements.
<code>dataengine/adjust.py</code>	Corporate-action back-adjustment (latest bar = current price).
<code>dataengine/merge.py</code>	Stitch multi-ISIN regimes into one continuous series.
<code>dataengine/funda.py</code>	BSE fundamentals: financials, shareholding, announcements.
<code>dataengine/marketcap.py</code>	Fill <code>shares_outstanding</code> from NSE/BSE.
<code>dataengine/export.py</code>	Render the compact static JSON bundle.
<code>dataengine/run.py</code>	CLI entrypoint: <code>nightly</code> , <code>backfill</code> , <code>eod</code> , <code>export</code> , <code>status</code> , <code>inspect</code> , <code>selftest</code> .

Dependencies (`backend/requirements.txt`): `fastapi`, `uvicorn[standard]`, `requests`, `curl_cffi` (browser-TLS fingerprinting for exchange APIs; graceful fallback to `requests`), and `firebase-admin` (push). Indicators use no `numpy/pandas` — all hand-rolled in `stdlib` for portability.

3. Data Sources (NSE / BSE) & Bhavcopy Parsing

The engine fetches exclusively from the official exchanges. EOD price data comes from the daily **bhavcopy** in UDiFF (standardized CSV) format; corporate actions and fundamentals come from exchange JSON APIs that sit behind bot-management (Akamai), requiring cookie priming.

Source	Endpoint / file	Used for
NSE bhavcopy (UDiFF)	nsearchives.../BhavCopy_NSE_CM_0_0_0_{ymd}_F_0000.csv.zip	Daily OHLCV (zip)
NSE legacy bhav	sec_bhavdata_full (fallback, no ISIN — symbol join)	Fallback OHLCV
NSE equity master	EQUITY_L.csv (ISIN, SYMBOL, SERIES, FACE_VALUE, listing date)	Security master
NSE corp actions	api/corporates-corporateActions?from..to (splits/bonus/div)	Adjustments
NSE quote-equity	api/quote-equity?symbol= (issuedSize)	Shares outstanding
BSE bhavcopy (UDiFF)	bseindia.com/.../BhavCopy_BSE_CM_0_0_0_{ymd}_F_0000.CSV (raw)	Daily OHLCV
BSE scrip list	bulk JSON: ISIN, SCRIP_CD, Mktcap (cr), segment	Cap + code enrichment
BSE fin results	quarterly P&L by scrip code	Financials
BSE shareholding	promoter / pledge / FII / DII %	Ownership
BSE announcements	filings, board meetings, order wins	Reference

Bhavcopy field mapping (UDiFF, case-insensitive)

`_parse_udiff()` defensively maps column aliases and keeps **cash equities only** (series EQ, BE, BZ, SM, ST, A, B, T, X, XT; instrument type STK). Critically, it reads `ClsPric` (**official close**), *not* `LastPric` — on illiquid venues the last tick can be stale, but the exchange's official weighted close is consistent. Each parsed row becomes a frozen `PriceRow(isin, symbol, date, open, high, low, close, volume, exchange, name)`.

```
ISIN      <- ISIN | ISIN NUMBER
TckrSymb  <- SYMBOL
FinInstrmNm <- SC_NAME | SECURITY      (company name)
SctySrs   <- SERIES | Sgmt        (EQ/BE/BZ/SM/ST/...)
FinInstrmTp <- STK                (stock only)
OpnPric/HghPric/LwPric/ClsPric <- OPEN/HIGH/LOW/CLOSE (ClsPric = official close)
TtlTradgVol <- TOTTRDQTY | TTL_TRD_QNTY
```

Corporate actions are parsed from the NSE feed in ≤365-day windows. Regexes extract split ratios (“10 -> 2”, “FV 10 to 5” → ratio 0.2), bonus ratios (“1:1” → 0.5, “2:1” → 0.333), and cash dividends (kept in a separate table). When a scrip changes ISIN (e.g. a post-merger or face-value split), the feed reports the old ISIN but the **symbol is canonical**, so ingest resolves symbol → live ISIN first.

4. The SQLite Spine — Schema

All state lives in \$TAUREYE_DATA_DIR/market.db, created by dataengine/db.py. Every ingest is idempotent (UPSERT / ON CONFLICT), so re-running a date never duplicates. Raw OHLC is preserved untouched; adj_close is the canonical price indicators read.

Core tables

```
securities (PK isin)
  isin, name, nse_symbol, bse_code, bse_symbol, series, sector,
  face_value, listed_on, shares_outstanding, shares_src, shares_at,
  segment (EQ|ETF|SME), about, active (0 = ISIN superseded), updated_at
  INDEX ix_sec_nse(nse_symbol), ix_sec_bse(bse_code)

prices (PK isin, date)
  open, high, low, close, volume,          -- raw, unadjusted
  adj_factor REAL DEFAULT 1.0,             -- multiplicative (<1 split, >1 bonus)
  adj_close REAL,                          -- close * adj_factor (4dp) <- golden
  source ("NSE" | "BSE" | "NSE+BSE")      -- NSE+BSE = cross-verified
  INDEX ix_prices_date(date)

corporate_actions (PK isin, ex_date, kind)  kind = split|bonus, ratio, detail, source
dividends      (PK isin, ex_date)           amount (Rs/share) -- NEVER fed to adjustment
quarantine      (isin, date, reason, payload JSON, created_at) -- recon mismatches
ingest_runs     (id, trade_date, stage, status, rows, quarantined, message, started/ended)
```

Fundamentals tables

```
financials (PK isin, period_end, period_type, consolidated)
  revenue, other_income, expenses, ebitda, interest, depreciation,
  pbt, tax, net_profit, eps, raw(JSON), source, updated_at

balance_sheets (PK isin, period_end, period_type)
  total_assets, total_liab, total_debt, long_term_debt, short_term_debt,
  cash, net_debt, equity, net_working_capital, raw, source, updated_at

shareholding (PK isin, period_end)
  promoter_pct, promoter_pledge_pct, public_pct, fii_pct, dii_pct, raw, source

announcements (PK isin, dt, headline) category, url, source
```

Anchoring rule. Because the spine stores *shares* (not a frozen market cap) and adj_close is always close × adj_factor, market cap and every indicator automatically track the latest close. The active=0 flag retires superseded ISINs so the exporter never emits duplicate candle files.

5. Ingest Pipeline (nightly orchestration)

`ingest.py :: run_nightly(trade_date)` runs eight independent stages; a failure in one does not stop the others (status degrades to *partial*). Every stage logs to `ingest_runs` and the text engine log.

#	Stage	What it does
1	<code>update_master()</code>	Fetch NSE EQUITY_L; upsert securities; retire superseded ISINs (active=0).
2	<code>update_corporate_actions()</code>	Fetch CA feed (~460d window); upsert; immediately readjust affected ISINs.
3	<code>update_dividends()</code>	Parse cash dividends from the same feed into the separate dividends table.
4	<code>ingest_eod(date)</code>	Fetch NSE + BSE bhavcopy; reconcile; insert golden rows into prices; quarantine.
5	<code>readjust()</code>	Recompute <code>adj_factor</code> / <code>adj_close</code> across full history for affected (or all) ISINs.
6	<code>update_segments()</code>	Classify EQ / ETF / SME via ISIN prefix + NSE list + symbol heuristics.
7	<code>update_sectors()</code>	Map NSE symbol -> sector from sectoral-index constituents (free, no vendor).
8	<code>enrich_shares()</code>	Fill <code>shares_outstanding</code> from NSE <code>issuedSize</code> / BSE bulk.

The “latest trading day” helper handles publication timing: NSE posts bhavcopy ~19:00 IST, so a weekday run before 19:00 uses the previous session, and weekends roll back to Friday. The production cron fires at **21:00 IST** (15:30 UTC), after close.

EOD insert (golden rows)

```
nse = _fetch_bhav("NSE", ...); bse = _fetch_bhav("BSE", ...) # UDiFF, legacy fallback
if not nse and not bse: return {"status": "error"}           # both down -> error
_upsert_bse_master(conn, bse); _upsert_nse_master(conn, nse_fixed)
golden, quarantine = reconcile(nse_fixed, bse)               # cross-verify
for r in golden:
    conn.execute("""INSERT INTO prices(isin,date,open,high,low,close,volume,
        adj_factor,adj_close,source) VALUES (?, ?, ?, ?, ?, ?, 1.0, ?, ?)
        ON CONFLICT(isin,date) DO UPDATE SET ...""",
        (r.isin, r.date, r.open, r.high, r.low, r.close, r.volume, r.close, r.exchange))
```

6. Corporate-Action Adjustment Algorithm

`adjust.py` back-adjusts history so the latest bar always equals current market price (CMP). A candle on date D is multiplied by the product of ratios of all actions whose ex-date is **after** D . Actions not yet gone ex (`ex_date > latest`) are ignored — applying them would move CMP off the last traded price (a cardinal rule).

```
def adjustment_factors(dates, actions):    # dates sorted ascending (ISO)
    factors = {d: 1.0 for d in dates}
    latest = dates[-1]
    for ca in actions:
        if ca.ex_date > latest: continue    # not yet effective -> ignore
        for d in dates:
            if d < ca.ex_date:              # only bars BEFORE the action
                factors[d] *= ca.ratio
    return factors

def apply_adjustments(rows, actions):
    rows = sorted(rows, key=lambda r: r.date)
    f = adjustment_factors([r.date for r in rows], actions)
    return [{ "date": r.date,
              "open": round(r.open * f[r.date], 4),
              "high": round(r.high * f[r.date], 4),
              "low": round(r.low * f[r.date], 4),
              "close": r.close,                # raw close preserved
              "adj_factor": f[r.date],
              "adj_close": round(r.close * f[r.date], 4) } ... ] # indicators read this
```

Worked example — stock at ■1000 on Jan 1, then a 1:1 bonus (ratio 0.5) ex Jan 15 (price halves to ~■500):

Date	Raw close	Factor	adj_close	Note
2026-01-01	1000	0.5	500	before bonus -> scaled to post-bonus level
2026-01-15	500	1.0	500	ex-date bar unchanged
2026-02-01	550	1.0	550	after bonus -> real CMP

The whole series is now on one continuous scale, so RSI/MACD/SMA never see a phantom gap, and the newest bar is the genuine current price.

7. NSE/BSE Reconciliation & Multi-ISIN Merge

Reconciliation

A scrip often trades on both venues. `reconcile.py` joins by ISIN and cross-checks OHLC (volume legitimately differs). Agreement test: within $\blacksquare 0.05$ absolute (`RECON_TOLERANCE_ABS`) or 0.5% relative (`RECON_TOLERANCE_PCT`). Outcome per ISIN, exactly once:

- **Agree** → one canonical row tagged `source="NSE+BSE"` (cross-verified).
- **Disagree** → keep NSE (authoritative/liquid) as golden; log the mismatch to quarantine with a per-field `{nse, bse}` payload.
- **NSE-only** or **BSE-only** → that venue's row is kept as-is.

Multi-ISIN merge (at export/read time)

When a security's ISIN changed, history is split across an old and a new ISIN. `merge.py` stitches them: the live (current) regime is kept verbatim, and each older regime is scaled by the **median ratio** over overlapping dates (`overlap_factor()`), then back-filled for dates the live regime lacks. Live bars always win on overlap.

```
# old regime closes 200/220; new regime opens 440 on the overlap date
overlap_factor: live=440 / old=220 -> k = 2.0
merged: 2025-01-01 close = 200 * 2.0 = 400 (old, scaled)
        2025-06-16 close = 440 (live, untouched)
        2026-06-12 close = 460 (live)
-> one continuous series at the current price scale
```

8. Server-Side Indicator Computation

`indicators.py :: compute_metrics(sec, candles)` computes one Metrics row per scrip from adjusted candles. All formulas are stdlib. The same logic is mirrored client-side for the legacy/live path, but in production these values are precomputed nightly and embedded in `metrics.json`.

Indicator	Periods	Formula / rule	Output field(s)
SMA	20/50/200	$\text{sum}(\text{last } p \text{ closes}) / p$	<code>sma_20/50/200</code> , <code>pct_above_sma*</code>
EMA	12/26/9	$k=2/(p+1)$; $\text{EMA} = v*k + \text{prev}*(1-k)$	(internal, feeds MACD)
RSI	14	Wilder-smoothed; $100 - 100/(1+RS)$	<code>rsi_14</code>
MACD	12/26/9	EMA12-EMA26, <code>signal=EMA9</code> , <code>hist=line-signal</code>	<code>macd_hist</code> (+cross flags)
ATR %	14	$\text{mean TR} / \text{close} * 100$	<code>atr_pct</code>
52w hi/lo	252 bars	$\text{max}(\text{high})/\text{min}(\text{low})$ over window	<code>dist_52w_high/low_pct</code> , <code>new_*</code>
Volume	20	$\text{vol} / \text{SMA}(\text{vol}, 20)$	<code>rel_volume</code> , <code>volume_spike</code> ($\geq 2x$)
Change	1	$(\text{close}/\text{prev}-1)*100$	<code>change_pct</code> , <code>change_abs</code> , <code>gap_up/down</code>
Market cap	static	$\text{close} * \text{shares} / 1e7$ (Rs cr)	<code>market_cap_cr</code>

Signal flags (1 if triggered today, else 0) are derived by comparing an indicator today vs one bar back: `golden_cross` / `death_cross` (50×200), `cross_20_50_up/down`, `macd_bull_cross/bear_cross` (hist crosses zero), `new_high_52w/new_low_52w`, `gap_up/gap_down` (open vs prev close $\pm 2\%$), and `volume_spike`.

```
def rsi(closes, period=14):
    if len(closes) <= period: return 50.0
    gains = losses = 0.0
    for i in range(1, period+1):
        ch = closes[i]-closes[i-1]
        gains += ch if ch >= 0 else 0; losses += -ch if ch < 0 else 0
    ag, al = gains/period, losses/period
    for i in range(period+1, len(closes)):
        ch = closes[i]-closes[i-1]
        ag = (ag*(period-1) + max(ch,0))/period
        al = (al*(period-1) + max(-ch,0))/period
    return 100.0 if al == 0 else 100 - 100/(1 + ag/al)
```

9. The Export Step — JSON Bundle Format

`export.py` renders the spine into compact, gzip-friendly JSON written to the nginx web root. This bundle is the production “backend”.

File	Shape (top-level keys)	Purpose
<code>metrics.json</code>	<code>meta</code> , <code>fields[]</code> , <code>segments[]</code> , <code>metrics[]</code>	Screener universe (~5,800 rows)
<code>fundamentals.json</code>	<code>meta</code> , <code>securities[] {s,n,x,sec,g,fv,so,isin}</code>	Search, cap segment, identity
<code>indices.json</code>	<code>indices[] {key,label,value,change_pct,category}</code> , <code>as_of</code> , <code>data_date</code>	Ticker / indices page
<code>ohlcv.json</code>	<code>dates[]</code> , <code>series[] {s,d[],o[],h[],l[],c[],v[]}</code>	Windowed OHLC (columnar, ~260d)
<code>candles/<SYM>.json</code>	<code>{s, adjusted, cols, candles[[date,o,h,l,c,v]]}</code>	Full per-symbol history (lazy)
<code>funda/<SYM>.json</code>	<code>corporate_actions[]</code> , <code>financials[]</code> , <code>shareholding[]</code> , <code>announcements[]</code>	Report page
<code>dividends.json</code>	<code>{dividends: {SYM: [[ex_date, amount]]}}</code>	Portfolio total-return
<code>reports/<SYM>.json</code>	<code>{report, analysis, disclaimer, generated_at}</code>	Nightly factual report
<code>manifest.json</code>	<code>files{}</code> with raw/gz sizes + counts	Bundle index

`metrics.json` — the row shape

```
{ "meta": { "generated_at": "...", "count": 5833 },
  "fields": [ { "key": "close", "label": "LTP", "group": "Price", "type": "number", "unit": "Rs", ... },
  "segments": [ { "key": "EQ", "label": "Equity", "count": 5200 }, { "key": "ETF", ... }, { "key": "SME", ... } ],
  "metrics": [ {
    "symbol": "RELIANCE", "name": "Reliance Industries", "exchange": "NSE", "sector": "Energy",
    "close": 2987.5, "change_pct": 0.52, "change_abs": 15.5, "volume": 18500000, "rel_volume": 1.23,
    "rsi_14": 58.3, "macd_hist": 2.15, "sma_20": 2950.25, "sma_50": 2920.0, "sma_200": 2850.75,
    "pct_above_sma20": 1.25, "pct_above_sma50": 2.32, "pct_above_sma200": 4.78,
    "dist_52w_high_pct": -8.5, "dist_52w_low_pct": 42.3, "atr_pct": 1.95,
    "market_cap_cr": 1787500, "segment": "EQ",
    "golden_cross": 0, "death_cross": 0, "cross_20_50_up": 0, "cross_20_50_down": 0,
    "new_high_52w": 0, "new_low_52w": 0, "macd_bull_cross": 0, "macd_bear_cross": 0,
    "gap_up": 0, "gap_down": 0, "volume_spike": 0 }, ... ] }
```

Per-symbol candle files keep adjusted O/H/L (raw × factor) and `c = adj_close`; `v` is raw volume; `adjusted:true` documents the contract.

10. HTTP Layer, CLI & Configuration

Robust fetch

http.py wraps a persistent session with cookie priming (prime_nse(), prime_nse_quotes() for Akamai), exponential backoff (retries on 429 / 5xx / connection errors), on-disk caching keyed by name, and **byte validation** predicates (is_zip, not_html) — a 200 whose body is an HTML error page is retried, and poisoned cache entries are dropped.

CLI (python -m backend.app.dataengine.run ...)

Command	Effect
nightly [--date]	Full pipeline for the latest (or given) trading day.
backfill [--days N]	Refresh master+CA once, then EOD for N trading days, then full readjust.
eod --date / master / corp-actions / readjust / segments / sectors	Individual stages.
marketcap / funda [--all --limit]	Enrich shares / fundamentals.
export [--all --gz --out PATH] [--metrics --ohlcv --candles --funda ...]	Publish JSON bundle.
status / inspect <SYMBOL> / logs	Diagnostics (DB counts, ISIN routing, log tail).
selftest	Offline validation of reconcile + adjustment math.

Config & env (config.py): TAUREYE_DATA_DIR, TAUREYE_DB_PATH, TAUREYE_PROVIDER (db|synthetic), TAUREYE_API_KEY; HTTP_TIMEOUT 30s, HTTP_RETRIES 4, HTTP_BACKOFF 1.6; reconciliation tolerances as above; FUNDA_SLEEP 0.4s, FUNDA_MAX_AGE_DAYS 7.

11. Frontend Data Layer (source switch, snapshot, client)

Three files define how the SPA gets data: `src/data/source.ts` (switch + URL helpers), `src/data/snapshot.ts` (bundle reader), and `src/api/client.ts` (the facade that branches on `LOCAL_DATA`).

```
// src/data/source.ts
export const LOCAL_DATA = import.meta.env.VITE_DATA_SOURCE === "local";
export const BUNDLED_BASE = `${import.meta.env.BASE_URL}data`; // in-app /data
const RAW_BASE = (import.meta.env.VITE_DATA_BASE ?? "").trim().replace(/\/$/, "");
export const DATA_BASE = RAW_BASE || BUNDLED_BASE; // remote host or bundled
export const CANDLE_BASE = (import.meta.env.VITE_CANDLE_BASE || "").trim() || DATA_BASE;
export const dataUrl = (n) => `${DATA_BASE}/${n}`;
export const candleUrl = (s) => `${CANDLE_BASE}/candles/${encodeURIComponent(s)}.json`;
```

Offline-first fetch. `snapshot.ts` tries the (possibly remote) `DATA_BASE` first so the app auto-updates from the host, then falls back to the in-app bundled copy so an unreachable host degrades to the last shipped snapshot rather than failing. Each bundle file is fetched once per session and cached as a **promise** (dedupes concurrent callers); failures drop the cache so a retry re-fetches. Candle files cache per-symbol. A defensive de-dup strips repeated symbol|exchange rows on load.

The client facade — every method branches on `LOCAL_DATA`

api method	LOCAL_DATA = true	LOCAL_DATA = false
health()	count metrics.json rows	GET /api/health
fields() / segments()	from metrics.json	GET /api/fields, /api/segments
screen(req)	runScreen() in-memory	POST /api/screen
metrics(sym)	find row in metrics.json	GET /api/metrics/{sym}
candles(sym,limit)	candles/<sym>.json (cached)	GET /api/candles/{sym}
searchSecurities(q)	rank fundamentals.json	GET /api/securities
indices()	indices.json (fail-soft empty)	GET /api/indices
dividends()	dividends.json -> Map	empty Map
stockReport / aiAnalysis	reports/<sym>.json (+ optional live)	GET /api/ai/analysis/{sym}
list/save/deleteScreen	localStorage (max 200)	GET/POST/DELETE /api/screens

The legacy `req()` helper retries the boot race (`TypeError / 502 / 503 / 504`, exponential backoff, 8 attempts) and injects an optional `X-API-Key`; genuine `4xx/5xx` throw immediately.

12. Type System & Client Screening Engine

src/api/types.ts defines the shared contract. The Metrics row carries price/volume/momentum/trend/range/volatility fields plus the integer signal flags. FieldDef drives the screener UI (key, label, group, unit, desc).

```
interface ScreenRequest {
  filters: Filter[]; logic: "AND"|"OR"; exchange?: "NSE"|"BSE"|null;
  segments?: string[]|null; query?: string|null;
  sort_by: string; sort_dir: "asc"|"desc"; limit: number;
}
interface Filter {
  field: string; op: "gt"|"gte"|"lt"|"lte"|"eq"|"between";
  value: number; value2?: number|null; join?: "AND"|"OR"|null; op2?: Operator;
}
interface ScreenResponse { count: number; total_universe: number; results: Metrics[]; }
```

src/lib/screenLocal.ts :: runScreen() is a faithful TS port of the Python screener, so web and APK produce identical results: pre-filter by exchange / segment / text query, apply metric filters with screen-level AND/OR, then sort with **missing values always sinking to the bottom** in both directions. Unknown fields are ignored; null values fail numeric comparisons.

```
rows = rows.filter(m => req.logic === "AND"
  ? req.filters.every(f => passes(m, f))
  : req.filters.some(f => passes(m, f)));
// passes(): per-field compare(); a filter with join+op2 ANDs/ORs its two conditions.
// sort: missing (null/NaN) -> always last; else va<vb ? -1 : 1, negated when desc.
```

13. Technical Indicators (formulas & periods)

Indicators are computed once (server, nightly) and read from `metrics.json`; the chart additionally draws moving-average overlays client-side (9 / 20 / 50 / 100 / 200; default 20/50/200).

Field	Definition
close / change_pct / change_abs	LTP; $(\text{close}/\text{prev}-1)*100$; close-prev.
sma_20/50/200	Simple moving averages of adjusted close.
pct_above_sma20/50/200	$(\text{close}/\text{sma} - 1) * 100$ -> distance above(>0)/below(<0) each MA.
rsi_14	Wilder RSI, 14d, 0-100 (>70 overbought, <30 oversold).
macd_hist	EMA12-EMA26 minus EMA9 signal (>0 bullish momentum).
atr_pct	Average True Range / close * 100 (typical daily swing).
dist_52w_high_pct / low_pct	% below 52w peak (0 = new high) / % above 52w trough.
rel_volume	today volume / 20-day average (2 = 2x normal).
market_cap_cr	close * shares_outstanding / 1e7 (Rs crore; null if shares unknown).

Signal flags usable as = 1 filters: `golden_cross`, `death_cross`, `cross_20_50_up`, `cross_20_50_down`, `new_high_52w`, `new_low_52w`, `macd_bull_cross`, `macd_bear_cross`, `gap_up`, `gap_down`, `volume_spike`. Gaps use a $\pm 2\%$ open-vs-prev-close threshold; a volume spike is $\geq 2\times$ the 20-day average; the 52-week window is 252 bars.

14. Chart-Pattern Detection

src/lib/patterns.ts :: detectPatterns(bars) runs rule-based, **descriptive** (non-advisory) detection on daily bars (min 30). It returns labels+details, chart markers, support/resistance levels, and white polylines tracing each shape. Helpers: SMA, pivot high/low (strict local extreme within $\pm k=5$ bars), least-squares `fit()` for trendlines, and Lagrange `quad()` for the cup parabola.

Pattern	Window	Key conditions
50/200 MA cross	25 bars	50-SMA crosses 200-SMA (either direction).
52-week high / low	252 bars	Today's high/low at or beyond the prior 252-bar extreme.
Double top (M)	90 bars	2 peaks within 3%, valley $\geq 4\%$ below, ≥ 8 bars apart, price below peaks.
Double bottom (W)	90 bars	Inverse of double top (troughs / crest).
Triangle (asc/desc/sym)	70 bars	Fitted high/low lines converge (start gap $> 1.3\times$ end gap); kind by slopes.
Cup & handle	140 bars	Parabolic base, depth 12-50%, rims within 6%, centered bottom, shallow handle.
Flag	28 bars	18%+ pole, then consolidation $\leq 60\%$ of pole range, near-flat upper rail.
Support / resistance	all	Horizontal lines at the most recent pivot low / pivot high.
Candlesticks	2 bars	Engulfing, Doji (body $\leq 10\%$ range), Hammer, Shooting star.
Compliance posture. Patterns are stated as factual geometry — no buy/sell labels, targets, or recommendations — consistent with the app's SEBI-aware, non-advisory stance (stock pages stay factual with a disclaimer).		

15. Natural-Language Screener

`src/lib/nlScreen.ts :: parseScreen(text)` is a deterministic, offline regex rule-engine (no LLM) that maps plain English to `Filter[]`. It lowercases + space-pads input, applies each rule (first match per `field:op` wins), detects segment hints, and chooses AND vs OR. It returns recognized phrases for UI feedback.

Phrase(s)	Filter produced
golden cross / death cross	golden_cross=1 / death_cross=1
20 sma crossing above/below 50	cross_20_50_up=1 / cross_20_50_down=1
new 52-week high / low	new_high_52w=1 / new_low_52w=1
oversold / overbought	rsi_14 < 30 / rsi_14 > 70
rsi below N / rsi above N	rsi_14 < N / rsi_14 > N
bullish / bearish macd	macd_bull_cross=1 / macd_bear_cross=1
above/below 200 50 20 dma	pct_above_sma200 50 20 > 0 / < 0
price above/below N	close > N / close < N
up N% / down N%	change_pct > N / change_pct < -N
gap up / gap down	gap_up=1 / gap_down=1
volume spike / high volume	volume_spike=1
large / mid / small cap	market_cap_cr >20000 / >=5000 / <5000
no etf / equity only	segments = ["EQ"]
...or... / any of...	screen logic = OR (else AND)

Example: *“golden cross and rsi below 30”* → [{golden_cross=1}, {rsi_14<30}], logic AND.

16. Screener Filter Model & Report Building

The `FIELDS` catalog (`server screener.py`; mirrored in the bundle's `fields[]`) groups ~24 filterable fields into Price, Volume, Momentum, Trend, Range, Volatility, and Signals. Operators: `gt`, `gte`, `lt`, `lte`, `eq`, `between`. A single filter can carry two conditions (`op+op2` joined by `AND/OR`), e.g. RSI between 30 and 70; the screen as a whole then combines filters by its own `AND/OR` logic. Sorting accepts any field plus `symbol/name`; invalid keys fall back to `market_cap_cr`; missing values always sort last.

Report building

`src/lib/reportData.ts :: buildAiReport(symbol)` assembles a factual per-stock report: it fetches corporate actions from `funda/<SYM>.json`, prefers the nightly cached `reports/<SYM>.json` (instant, free), and in optional live mode can generate one via a Supabase Edge Function using metrics + detected patterns + corp actions. `ReportView.tsx` renders it (commentary, fundamentals, corp actions, ratios) with a print-to-PDF stylesheet.

Shareable screens & watchlists

Screens and watchlists serialize to URL tokens — `/app/screener?s=<token>` and `/app/watchlist?w=<token>` — encoded client-side with `lz-string` compression (with a legacy `base64url` fallback for older links), so sharing needs no server storage and fits the static model. Opening a watchlist link imports its scrips into a new local list.

17. Frontend UI — Routing, Shell, Pages, Components

Entry (`src/main.tsx`): `StrictMode` → `ErrorBoundary` → `BrowserRouter` → `AuthProvider`; theme/settings applied before first paint; on native it initializes Cappo OTA, AdMob, and the OAuth deep-link handler. Routing (`src/App.tsx`) splits public vs protected:

Scope	Routes
Public	/ (Landing on web / Login on native), /login, /reset-password, /i/:code (invite), /legal/:doc, /about, /contact, /blog, /blog/:slug
Protected /app/* (RequireAuth + BootGate)	dashboard, screener, chart, indices, alerts, watchlist, portfolio, calculators, saved, wallet, refer, settings

`RequireAuth` waits for the session to hydrate (so a refresh doesn't bounce a logged-in user), then redirects unauthenticated users to `/?next=<path>` so a shared deep link survives login. `BootGate` preloads data before the shell renders.

App shell & pages

`AppShell.tsx` provides the top bar (brand cluster with home+alerts, responsive hamburger drawer, theme toggle, credits badge, global scrip search, sign-out) and the index ticker, rendering nested routes via `<Outlet/>`. Key pages: **Screener** (filters, presets, NL builder, save/share), **Chart** (lightweight-charts candles, MA overlays, pattern drawings, alerts, report), **Watchlist** (multiple local lists, since-add change, import shared), **Portfolio** (holdings P&L, diversification, dividends), **GlobalIndices** (domestic / international / FX / depository, TradingView links), **Dashboard** (snapshot + sector heatmap), **Calculators**, **Saved**, **Wallet**, **Refer**, **Settings**, plus public **Landing / Login / About / Contact / Blog / Article / Legal**.

Reusable components include `AuthPanel`, `ReportView`, `PublicLayout`, `IndexTicker`, `WatchlistMenu` (flips above the anchor when clipped), `ScripSearch`, `Markdown` (zero-dep renderer), `BullScene` (lazy three.js, web only), and `ErrorBoundary`.

18. Client State & Persistence; Content System

User data is device-local (localStorage) with change events for cross-tab/-component sync; cloud credits/referrals use Supabase RPC when configured.

Concern	localStorage key	Shape (essentials)
Watchlists	taureye.watchlists	[[{id,name,items:[{symbol,name,exchange,addedAt?,addedPrice?}]]]
Alerts	taureye.alerts.v1	[[{symbol,name,exchange,level,type:above below,active}]]
Portfolios	taureye.portfolios.v1	[[{id,name,positions:[{symbol,qty,avgBuyPrice,buyDate}]]]
Saved screens	taureye.screens.local	[[{id,name,request,created_at}]] (max 200)
Settings	taureye.settings.v1	{scripColor?,upColor?,downColor?,accentColor?}
Session (offline)	taureye.session	{id?,name,email}
Referral pending	taureye.referral.pending.v1	captured ?ref= code, claimed post-signup
Credits ledger	taureye.credits.ledger.v1	append-only {id,ts,delta,reason}

Content system. `src/content/articles.ts` holds the blog as data —

`{slug,title,summary,category,date,readMins,body}` with Markdown bodies — rendered by the line-based `src/lib/markdown.tsx` (headings, lists, **bold**/*italic*/code, internal/external links). Public pages use `PublicLayout` and are reachable without login.

Credit economy (`src/lib/economy.ts`) is feature-flagged `ECONOMY_ENABLED=false` — all costs/rewards are 0 and it is dormant until turned on. **Theming** is CSS custom properties with a dark default and a `[data-theme=light]` override; the accent is monochrome (white on dark / black on light), with green reserved for gains and red for losses; users can override four colors in Settings.

19. Authentication (Supabase)

`src/lib/supabase.ts` creates the client only when both `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` are present (`isSupabaseConfigured`); otherwise the app runs guest/offline. PKCE flow, persisted sessions, auto-refresh.

```
export const supabase = isSupabaseConfigured
  ? createClient(url, anonKey, { auth: {
      persistSession: true, autoRefreshToken: true, flowType: "pkce" } })
  : null;
```

`AuthContext.tsx` exposes `signIn`, `signUp`, `oauth(provider)`, `resetPassword`, `resendConfirmation`, `updatePassword`, plus recovery state. It hydrates the session on mount and subscribes to `onAuthStateChange` (catching `PASSWORD_RECOVERY` to route to `/reset-password`). When unconfigured, all methods short-circuit to a local in-memory user persisted to `localStorage`.

OAuth — web vs native

Web uses a full-page redirect back to `/login` with Supabase's `detectSessionInUrl`. Native uses `signInWithOAuth({skipBrowserRedirect:true})`, opens the provider URL via `@capacitor/browser`, and the provider deep-links back to `app.taureye.mobile://auth-callback?code=...`; `main.tsx`'s `appUrlOpen` listener calls `exchangeCodeForSession(code)`. `AuthPanel.tsx` drives the multi-step email/password form, the verify-email and forgot-password sub-flows, the visible OAuth providers (`VITE_OAUTH_PROVIDERS`), referral capture, and the `?next=` post-auth redirect (stashed in `sessionStorage` so it survives the OAuth round-trip).

20. Mobile Platform (Capacitor + Capgo OTA)

capacitor.config.ts: appld app.taureye.mobile, webDir dist, HTTPS-only (cleartext:false, allowMixedContent:false), CapacitorHttp.enabled (native HTTP bypasses CORS/mixed-content for the data host), FirebaseMessaging, SplashScreen, and Capgo.

```
CapacitorUpdater: {  
  autoUpdate: "onLaunch",          // check + apply at cold start (same session)  
  appReadyTimeout: 10000,          // rollback if notifyAppReady() not called in 10s  
  defaultChannel: "production",  
  autoSplashscreen: true, autoSplashscreenTimeout: 15000 }
```

main.tsx calls CapacitorUpdater.notifyAppReady() on native (no-op on web); skipping it makes Capgo roll back to the last good bundle. src/lib/ota.ts surfaces OTA progress (checking / downloading / ready / uptodate / failed) to the loading screen. **OTA only updates the web layer** — native changes (new plugins, config, permissions, Capacitor version) require an APK rebuild + reinstall.

Native plugins: @capacitor/app (deep links), /browser (OAuth), /share (native share sheet), /local-notifications, /splash-screen, @capacitor-firebase/messaging (FCM push), @capacitor-community/admob (opt-in banners), @capgo/capacitor-updater (OTA). The android.yml workflow builds a debug APK (Node 22, JDK 21, Android SDK; web built in LOCAL mode with absolute VM/data URLs) and publishes it to a mobile-latest GitHub Release for sideloading.

21. Deployment & Infrastructure (CI/CD, Docker, nginx)

GitHub Actions

Workflow	Trigger	Effect
deploy.yml	push main	Build SPA (LOCAL, same-origin /data) -> rsync to nginx root EXCLUDING data/ -> reload nginx.
mobile-ota.yml	push main (web paths)	Build SPA (LOCAL, absolute VM URLs) -> upload to Capgo production channel.
android.yml	push main / manual	Build debug APK -> GitHub Release mobile-latest.
ios.yml	manual	iOS build.
refresh-data.yml	manual	Sync exporter + run refresh.sh on the VM (50-min cap), or diagnose.
dataops.yml	manual	Allow-listed VM data commands (status, backfill, dedupe, publish) without SSH.

Never clobber /data. The web deploy rsync always excludes data/ so it never overwrites the live JSON bundle. Data freshness is owned by the nightly cron, not by code deploys.

Container topology (Docker Compose on one VM)

Service	Image / base	Role
web	Dockerfile.web (node22 build -> nginx:1.27)	Serves SPA + /data (volume mount ./data/bundle, ro).
dataengine	Python backend image	Holds market.db (./data/spine); exports bundle to ./data/bundle.
scheduler	mcuadros/ofelia	Reads ofelia labels; fires nightly refresh+export at 15:30 UTC (21:00 IST).
supabase-*	official supabase/docker	Kong gateway, GoTrue auth, Postgres, Realtime, Studio.

deploy/azure/bootstrap.sh is an idempotent one-command setup: installs Docker, clones the Supabase self-host stack, generates secrets once (JWT_SECRET, Postgres/dashboard passwords, ANON_KEY/SERVICE_ROLE_KEY as HS256 JWTs), reconciles public URLs to the VM IP, applies the TaurEye SQL schema (schema, credits, referrals, subscriptions, harden-grants), seeds/builds the spine, starts everything, and exports the first bundle. nginx serves /data/ with CORS + 1h cache + gzip and falls back to index.html for SPA routes. harden.sh disables SSH password/root login, installs fail2ban, binds Kong to localhost, and adds nginx security headers; refresh.sh runs nightly under an flock single-writer guard.

npm scripts: dev (Vite :5174), build (tsc -b && vite build), lint, cap:sync, apk:debug. Production-style build sets VITE_DATA_SOURCE=local with VITE_DATA_BASE/VITE_CANDLE_BASE pointing at the host's /data.

22. Reproduction Blueprint & Checklist

To stand up an equivalent system end-to-end:

A. Data engine

- Build the SQLite spine schema (securities, prices, corporate_actions, dividends, financials, balance_sheets, shareholding, announcements, quarantine, ingest_runs).
- Implement source adapters for the official exchanges' EOD bhavcopy (UDiFF) + corporate-action and fundamentals APIs, with cookie/Akamai priming, retries, caching, and byte validation.
- Read the *official close*, not last-traded. Reconcile dual-venue prices with absolute+relative tolerances; quarantine disagreements; keep the authoritative venue.
- Back-adjust by corporate actions so the latest bar = current price; ignore not-yet-ex actions; merge multi-ISIN regimes via median overlap factor.
- Compute indicators + signal flags in plain code (SMA/EMA/RSI/MACD/ATR/52w/volume/gaps/crosses).

B. Static bundle + hosting

- Export compact JSON: metrics, fundamentals, indices, ohlc, per-symbol candles + funda + reports, dividends, manifest (gzip).
- Serve the built SPA + the bundle under /data via nginx (CORS, cache, gzip, SPA fallback). Run the exporter nightly via a scheduler; never let deploys overwrite /data.

C. Frontend

- One build-time data-source switch; a facade that branches every call between the bundle reader and a (optional) live API; offline-first fetch with in-app fallback and per-session promise caches.
- Port the screener (filters + AND/OR + sort with missing-last) to the client so web and mobile match. Add indicators read-through, rule-based pattern detection, and a regex NL-to-filter engine.
- Build the routed UI (public + auth-gated /app/*), device-local watchlists/portfolios/alerts/saved-screens, a data-driven blog, monochrome theming, and lz-string share links.

D. Auth, mobile, CI/CD

- Wire Supabase (PKCE; email/password + OAuth; recovery; guest fallback). Handle native OAuth via a deep-link code exchange.
- Wrap with Capacitor; deliver web updates over the air (Capgo, autoUpdate on launch, notifyAppReady gate); rebuild the APK only for native changes.
- CI: build SPA in LOCAL mode and (a) rsync to nginx excluding data/, (b) upload a bundle to the OTA channel, (c) publish a debug APK. Provision the VM with one idempotent bootstrap script; add a hardening pass.

This report describes the system as built. Field names, formulas, schemas, and topology above are taken directly from the source tree and are sufficient to reimplement each layer. Treat exchange endpoints and credentials as operational details to be supplied per deployment.